

Rediseño de la clase ProbNet

Motivaciones, decisiones y resultado

org.openmarkov.core — paquete model.network

Manuel Arias

Abril de 2026

Índice

1. Fundamentos de ingeniería del software	2
1.1. Principios de diseño orientado a objetos	2
1.2. Antipatrones detectados	3
1.3. Patrones de diseño aplicados	3
2. Situación de partida	4
2.1. La clase en cifras	4
2.2. Herencia directa de Graph<Node>	6
2.3. Campos reader y writer en el modelo de dominio	6
2.4. Bugs conocidos	6
2.5. Code smells	7
3. Decisiones de rediseño	7
3.1. Fase 1: corrección de bugs y simplificaciones	7
3.2. Extracción de NetworkMetadata	8
3.3. Extracción de ProbNetCopier	9
3.4. Fase 3: extracción de ProbNetClassifier	10
3.5. Fase 4: extracción de ProbNetAgentManager	11
3.6. Fase 5: extracción de ProbNetPotentialQueries	11
3.7. El cambio más importante: de herencia a composición	12
4. Resultado: arquitectura después del rediseño	15
4.1. Diagrama de la arquitectura actual	15
4.2. Las clases extraídas	17
4.3. Comparación de métricas	17
4.4. Qué no cambió	17
5. Orden de ejecución y riesgos	17

Resumen

ProbNet es la clase más crítica de OpenMarkov: **490 ficheros** la referencian en todo el proyecto. Con **1440 líneas** y más de **80 métodos públicos** distribuidos en seis áreas de responsabilidad distintas, la clase había acumulado más trabajo del que le correspondía.

Este documento describe *qué* se cambió, *por qué* se tomó cada decisión y *cuál es el resultado* en términos de arquitectura y métricas de código. El documento de análisis previo (*Análisis de la clase ProbNet*) recoge el diagnóstico detallado; aquí se parte de él y se documenta la ejecución.

1. Fundamentos de ingeniería del software

Antes de explicar los cambios concretos conviene definir los conceptos de ingeniería del software que se aplican a lo largo del documento. Esta sección es especialmente útil para lectores con poca experiencia en diseño orientado a objetos, ya que los términos técnicos se usan con precisión en el resto del texto.

1.1. Principios de diseño orientado a objetos

Los *principios de diseño* son reglas generales que, cuando se aplican con criterio, producen código más fácil de entender, modificar y probar. No son recetas infalibles, sino guías para tomar decisiones de diseño razonadas.

Principio de Responsabilidad Única (SRP). El *Single Responsibility Principle* establece que **una clase debe tener una sola razón para cambiar** [2]. Si un módulo hace demasiadas cosas a la vez, cualquier cambio en una de ellas obliga a revisar y probar todas las demás.

Principio DRY (*Don't Repeat Yourself*). El principio DRY establece que **cada pieza de conocimiento debe tener una única representación en el sistema** [4]. Cuando la misma lógica aparece en dos lugares, una corrección o un cambio de comportamiento debe aplicarse en ambos; si se olvida uno, el código queda inconsistente.

El ejemplo más directo en ProbNet era que el método `checkProbNet()` recorría la lista de restricciones con la misma lógica que `getUnsatisfiedConstraints()`, sin reutilizar ese último.

Principio de Inversión de Dependencias (DIP). El *Dependency Inversion Principle* establece que **los módulos de alto nivel no deben depender de los módulos de bajo nivel** [2]. En diseño orientado a objetos, esto se concreta en: el modelo de dominio (lo más importante en una aplicación) no debería conocer los detalles de los sistemas que lo rodean, como la entrada/salida de ficheros.

Si el modelo sabe cómo leer y escribir ficheros, un cambio en el formato de archivo puede obligar a modificar el modelo de dominio, que es la parte más delicada del sistema.

Composición frente a herencia. En programación orientada a objetos existen dos formas de reutilizar comportamiento: la *herencia* (“clase B es un tipo de clase A”) y la *composición* (“clase B *tiene* un objeto de tipo A”).

La herencia crea un acoplamiento muy fuerte: la subclase hereda *todo* lo de la clase padre, incluyendo lo que no necesita, y cualquier cambio en el padre puede romper la subclase. La composición es más flexible: el objeto contenido puede cambiarse o sustituirse sin tocar la clase contenedora.

Un principio clásico de diseño recomienda **preferir la composición frente a la herencia** siempre que la relación entre clases no sea genuinamente “es un tipo de” [3].

Principio de Segregación de Interfaces (ISP). El *Interface Segregation Principle* establece que **una clase no debe ser obligada a depender de métodos que no utiliza**. Es preferible tener varias interfaces pequeñas y específicas que una sola interfaz grande: así, cada consumidor declara exactamente las capacidades que necesita, sin verse arrastrado por métodos que nunca va a llamar.

En la práctica, esto significa que un algoritmo de inferencia que solo necesita consultar el grafo de la red no debería necesitar importar toda la clase **ProbNet** con sus 80 métodos; debería poder trabajar con una interfaz más pequeña que defina solo las operaciones de consulta.

1.2. Antipatrones detectados

Un *antipatrón* es un patrón recurrente de diseño que, en apariencia razonable, genera en la práctica problemas de mantenimiento o fragilidad. Reconocer los antipatrones en el código propio es el primer paso para corregirlos.

God Class (Clase Dios). **Descripción:** una clase que acumula demasiadas responsabilidades: lo sabe todo y lo hace todo. En lugar de delegar en otras clases, concentra lógica de distintos dominios en una sola unidad.

Por qué es dañina: viola directamente el SRP. Cualquier cambio requiere entender la clase entera, porque los distintos dominios están entrelazados. Las pruebas unitarias son difíciles de escribir por el elevado número de combinaciones posibles de estado. La God Class se convierte en el cuello de botella del desarrollo: todo pasa por ella.

Solución habitual: identificar las distintas responsabilidades y extraer cada una a su propia clase. Esto es exactamente lo que la técnica *Extract Class* persigue.

Herencia inapropiada. **Descripción:** usar herencia para reutilizar la implementación de otra clase, aunque la relación “es un tipo de” no sea semánticamente correcta. El resultado es que la subclase hereda métodos y campos que no le corresponden y queda atada a todos los cambios del padre.

Por qué es dañina: produce acoplamiento innecesario entre clases que conceptualmente son independientes. Además, dificulta el uso de interfaces: si una clase hereda de una clase concreta, el código cliente no puede desacoplarse fácilmente de la implementación.

Solución habitual: sustituir la herencia por composición y declarar las capacidades de la clase mediante interfaces.

1.3. Patrones de diseño aplicados

Un *patrón de diseño* es una solución reutilizable a un problema frecuente de diseño orientado a objetos [3]. A diferencia de los antipatrones, los patrones de diseño son soluciones contrastadas.

Extract Class (Extraer Clase). **Descripción:** cuando una clase tiene demasiadas responsabilidades, se identifica un grupo cohesivo de métodos y campos y se mueve a una nueva clase. La clase original delega en la nueva mediante llamadas explícitas.

Aplicación en este documento: se aplica en cinco ocasiones para reducir **ProbNet**: se extrae la clasificación de la red (**ProbNetClassifier**), la gestión de agentes (**ProbNetAgentManager**),

las consultas complejas de potenciales (`ProbNetPotentialQueries`), la lógica de copia (`ProbNetCopier`) y los metadatos descriptivos (`NetworkMetadata`).

Value Object (Objeto Valor). **Descripción:** un objeto que agrupa datos relacionados sin identidad propia: lo que importa es su contenido, no su dirección en memoria. Típicamente es mutable (porque sus datos pueden cambiar) pero no tiene lógica de negocio compleja.

Aplicación en este documento: `NetworkMetadata` agrupa los metadatos de la red (nombre, comentario, estados por defecto, criterios, etc.) en un único objeto que `ProbNet` posee. Al separar los datos de configuración del modelo de dominio en sí, queda claro qué es infraestructura descriptiva y qué es comportamiento.

Interfaces como contrato mínimo. **Descripción:** en lugar de hacer que todo el código cliente dependa de una clase concreta grande, se definen interfaces que describen solo las capacidades que cada cliente necesita. El cliente declara su dependencia sobre la interfaz; la clase concreta implementa todas las interfaces que corresponda.

Aplicación en este documento: se introducen las interfaces `GraphNetwork` y `PotentialNetwork` que permiten a los algoritmos de inferencia declarar exactamente qué vista de la red necesitan, desacoplándose del tipo concreto `ProbNet`.

2. Situación de partida

2.1. La clase en cifras

`ProbNet` era la clase más grande y más referenciada del proyecto. La Figura 1 muestra un diagrama UML que agrupa sus métodos por responsabilidad y señala los problemas detectados.

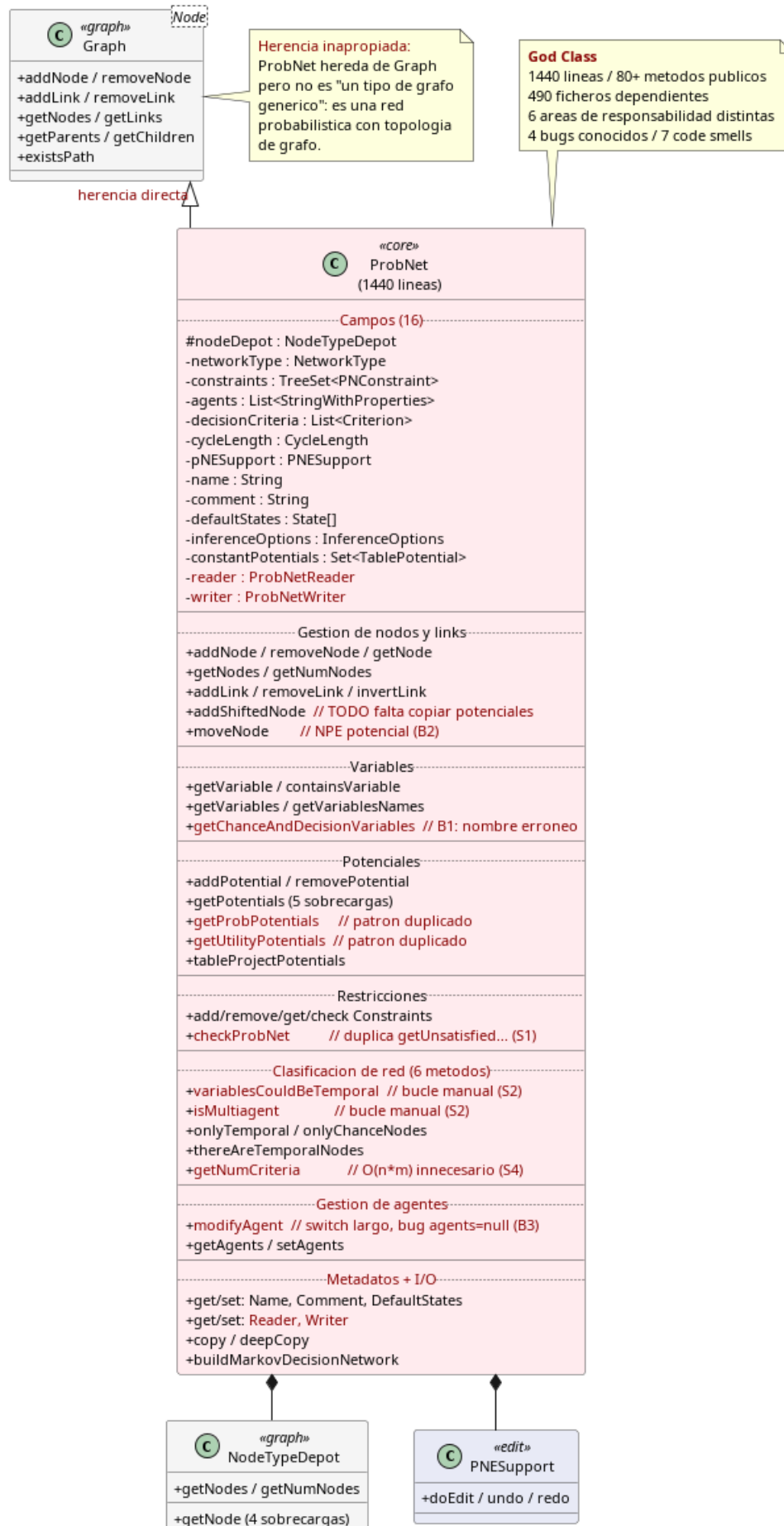


Figura 1: Diagrama UML de ProbNet antes del rediseño (1440 líneas). En **rojo** se señalan los problemas principales.

Las cifras más llamativas eran:

Métrica	Valor
Líneas de código	1 440
Métodos públicos	80+
Campos	16
Ficheros dependientes	490
Áreas de responsabilidad	6
Bugs conocidos	4
Code smells documentados	7
Cobertura de tests (métodos)	37 %

Las seis áreas de responsabilidad eran: gestión de nodos/links/variables, gestión de potenciales, restricciones de red, clasificación de la red, gestión de agentes y metadatos + I/O. Cada una de estas áreas responde a motivos de cambio distintos, lo que convierte a **ProbNet** en un ejemplo claro de God Class (§1.2).

2.2. Herencia directa de **Graph<Node>**

ProbNet extendía **Graph<Node>**, la clase genérica de grafo de la librería interna. Esta relación de herencia era inapropiada (§1.2): una red probabilística *usa* un grafo para representar su topología, pero no *es* un grafo genérico.

La consecuencia práctica era que cualquier código que quisiera desacoplarse de los detalles de **ProbNet** no podía, porque no existía ninguna interfaz intermedia. Un algoritmo de inferencia necesitaba importar la clase concreta aunque solo usara operaciones de consulta del grafo.

2.3. Campos **reader** y **writer** en el modelo de dominio

ProbNet almacenaba referencias a **ProbNetReader** y **ProbNetWriter**, los objetos responsables de la lectura y escritura de ficheros. Esto era una violación directa del DIP (§1.1): el modelo de dominio conocía los detalles del sistema de I/O.

El motivo por el que estos campos existían era funcional: la GUI necesitaba saber con qué lector/escritor se había abierto la red para usar el mismo formato al guardar. Sin embargo, esa información pertenece al controlador de ficheros, no al modelo. El acoplamiento no era necesario, y los campos permanecen en **ProbNet** de momento por compatibilidad con la GUI, pero ya están aislados dentro de **NetworkMetadata**.

2.4. Bugs conocidos

El análisis previo (véase *Análisis de la clase ProbNet*) identificó cuatro bugs:

Bug	Método	Descripción
B1	<code>getChanceAndDecisionVariables()</code>	El nombre indicaba que devolvía variables CHANCE y DECISION, pero el <code>switch</code> incluía también SV_PRODUCT y SV_SUM. Nombre engañoso o comportamiento incorrecto.
B2	<code>moveNode()</code>	<code>getNode(name)</code> puede devolver <code>null</code> si el nombre no existe; no había comprobación de nulidad antes de usarlo.
B3	<code>modifyAgent(REMOVE)</code>	Al eliminar el último agente asignaba <code>agents = null</code> . Cualquier llamada posterior a <code>getAgents()</code> causaba un <code>NullPointerException</code> .
B4	<code>addShiftedNode()</code>	Copiaba el tipo, coordenadas y propiedades del nodo, pero no sus potenciales. El propio código tenía un comentario <code>// TODO</code> reconociendo la omisión.

2.5. Code smells

Los siete code smells detectados cubrían: duplicación de lógica entre `checkProbNet()` y `getUnsatisfiedConstraints()` (violación DRY); bucles manuales donde existía un método más expresivo; duplicación del patrón de iteración en `getProbPotentials()` y `getUtilityPotentials()`; complejidad $O(n \cdot m)$ innecesaria en `getNumCriteria()`; código comentado en `addNodeConsistently()`; `toString()` colocado antes de los campos; y los campos `reader/writer` en el modelo de dominio.

3. Decisiones de rediseño

El rediseño se ejecutó en fases incrementales, verificando que todos los tests seguían pasando al final de cada una. El principio rector fue: **no romper la API pública existente** mientras se reduce el tamaño y la complejidad de ProbNet.

3.1. Fase 1: corrección de bugs y simplificaciones

Los cuatro bugs y los siete code smells se resolvieron sin cambiar la API pública:

Bug B1 — Renombrar `getChanceAndDecisionVariables()`. El análisis del uso en el proyecto confirmó que los tipos SV_PRODUCT/SV_SUM debían incluirse, porque los llamadores esperaban ese comportamiento. El problema era, por tanto, el nombre. Se renombró a `getNonUtilityVariables()`, que describe con precisión lo que el método hace: devuelve todas las variables que no son de utilidad. Se conservó el nombre antiguo como método `@Deprecated` para compatibilidad.

Bug B2 — Null-check en `moveNode()`. Se añadió una comprobación explícita: si `getNode(name)` devuelve `null`, el método ignora ese nombre en lugar de lanzar una excepción.

Bug B3 — Inicializar `agents` con lista vacía. Se eliminó la asignación `agents = null` en `modifyAgent(REMOVE)`. La lista se inicializa como vacía desde el principio y nunca se asigna `null`. Esto garantiza que `getAgents()` siempre devuelve una lista (posiblemente vacía), nunca `null`.

Bug B4 — addShiftedNode() sin potenciales. La resolución de este bug queda pendiente porque requiere decidir qué comportamiento esperan los llamadores: ¿copiar los potenciales del nodo original, o crear un potencial uniforme por defecto? Se añadió un test que documenta el comportamiento actual y el comentario @ToCheck.

Code smells S1–S7. checkProbNet() se redujo a una sola línea delegando en getUnsatisfiedConstraints() (S1, DRY). variablesCouldBeTemporal() e isMultiagent() pasaron a usar hasConstraintOfClass() (S2). getNumCriteria() usa ahora un TreeSet (S4, $O(n)$ en vez de $O(n \cdot m)$). Se eliminó el código comentado de addNodeConsistently() y se convirtió la cadena if en switch (S5). Se movió toString() al final del fichero (S6).

Los listados 1 y 2 ilustran la eliminación del smell S1.

```
1 // Codigo duplicado en checkProbNet():
2 public boolean checkProbNet() {
3     for (PNConstraint constraint : constraints) {
4         if (!constraint.isMetBy(this)) {
5             return false;
6         }
7     }
8     return true;
9 }
10
11 // La misma logica en getUnsatisfiedConstraints():
12 public List<PNConstraint> getUnsatisfiedConstraints() {
13     List<PNConstraint> unsatisfied = new ArrayList<>();
14     for (PNConstraint constraint : constraints) {
15         if (!constraint.isMetBy(this)) {
16             unsatisfied.add(constraint);
17         }
18     }
19     return unsatisfied;
20 }
```

Listado 1: Antes: dos métodos con la misma lógica (violación DRY).

```
1 public boolean checkProbNet() {
2     return getUnsatisfiedConstraints().isEmpty();
3 }
```

Listado 2: Después: checkProbNet() delega en getUnsatisfiedConstraints() (DRY).

3.2. Extracción de NetworkMetadata

Motivación (SRP, §1.1; Extract Class, §1.3; Value Object, §1.3): ProbNet almacenaba directamente todos los metadatos descriptivos de la red: nombre, comentario, si mostrar el comentario al abrir, estados por defecto, criterios de decisión, agentes, longitud de ciclo, opciones de inferencia y propiedades adicionales de formato. Estos datos no tienen comportamiento de dominio propio y cambiarán cuando cambie la interfaz de usuario o el formato de fichero, no cuando cambie la lógica de inferencia.

Decisión: se extrajeron los 11 campos de metadatos a un objeto valor NetworkMetadata que ProbNet posee como campo privado. ProbNet conserva todos sus getters y setters públicos, pero ahora delegan en NetworkMetadata. La API pública no cambia; solo la estructura interna.

Los listados 3 y 4 muestran la diferencia desde el punto de vista de ProbNet.

```
1 public class ProbNet extends Graph<Node> {
2     private String name;
```



```

3     private String comment = "";
4     private boolean showCommentWhenOpening = false;
5     private State[] defaultStates = { ... };
6     private InferenceOptions inferenceOptions = new InferenceOptions();
7     private CycleLength cycleLength = new CycleLength();
8     // ... 5 campos mas de metadatos ...
9
10    public String getName() { return name; }
11    public void setName(String name) { this.name = name; }
12    // ...
13 }

```

Listado 3: Antes: campo name directamente en ProbNet.

```

1 public class ProbNet implements PotentialNetwork, ... {
2     private final NetworkMetadata metadata = new NetworkMetadata();
3     // ... campos de dominio (grafo, restricciones, etc.) ...
4
5     public String getName() { return metadata.getName(); }
6     public void setName(String name) { metadata.setName(name); }
7     // ... los demas getters/setters delegan igual ...
8 }

```

Listado 4: Después: metadatos delegados a NetworkMetadata.

3.3. Extracción de ProbNetCopier

Motivación (SRP, §1.1; Extract Class, §1.3): ProbNet contenía dos métodos de copia, copy() y deepCopy(), cuya implementación era compleja (iterar nodos, links, potenciales y restricciones, y decidir qué se comparte y qué se clona en profundidad). Esta lógica no pertenece al modelo de dominio sino al proceso de preparación de la red para inferencia.

Decisión: se extrajo la lógica de copia a una clase de utilidad de paquete ProbNetCopier (visibilidad de paquete, no pública). Los métodos copy() y deepCopy() de ProbNet permanecen en la API pública como delegaciones:

```

1 // ProbNet.java (API publica inalterada):
2 public ProbNet copy() {
3     return ProbNetCopier.shallowCopy(this);
4 }
5
6 public ProbNet deepCopy() {
7     return ProbNetCopier.deepCopy(this);
8 }
9
10 // ProbNetCopier.java (logica compleja, fuera del dominio):
11 final class ProbNetCopier {
12     private ProbNetCopier() {}
13
14     static ProbNet shallowCopy(ProbNet original) {
15         // copia la estructura compartiendo referencias
16         // a Variables y Potentials (semantica "copy")
17         // ... 40 lineas de logica ...
18     }
19
20     static ProbNet deepCopy(ProbNet original) {
21         // crea nuevas instancias de todos los objetos mutables
22         // ... 50 lineas de logica ...
23     }
24 }

```

3.4. Fase 3: extracción de ProbNetClassifier

Motivación (SRP, §1.1; Extract Class, §1.3): ProbNet contenía seis métodos cuyo único propósito era clasificar la red según sus restricciones y contenido: `variablesCouldBeTemporal()`, `isMultiagent()`, `onlyTemporal()`, `onlyChanceNodes()`, `thereAreTemporalNodes()` y `getNumCriteria()`. Estos métodos son consultas puras: no modifican la red. Su único consumidor relevante era la GUI y los propios algoritmos de configuración.

Por qué como clase estática y no como métodos de instancia: los métodos de clasificación no necesitan acceder a ningún estado mutable de ProbNet; solo consultan restricciones y nodos. Una clase de utilidad estática (siguiendo el patrón ya establecido con `VariableTypeConverter` y `VariableStateOperations`) es la opción más honesta porque hace explícito que no hay estado y que los métodos son funciones puras.

Retrocompatibilidad: ProbNet conserva los seis métodos de instancia como delegaciones hacia `ProbNetClassifier.xxxx(this)`. El código existente que llame a `probNet.isMultiagent()` sigue funcionando sin ningún cambio.

```
1 // En ProbNet.java (retrocompatibilidad garantizada):
2 public boolean variablesCouldBeTemporal() {
3     return ProbNetClassifier.variablesCouldBeTemporal(this);
4 }
5
6 public boolean isMultiagent() {
7     return ProbNetClassifier.isMultiagent(this);
8 }
```

Listado 6: Delegación en ProbNet después de la extracción.

```
1 public final class ProbNetClassifier {
2
3     private ProbNetClassifier() {} // no instanciable
4
5     public static boolean variablesCouldBeTemporal(ProbNet probNet) {
6         return !probNet.hasConstraintOfClass(OnlyAtemporalVariables.class);
7     }
8
9     public static boolean isMultiagent(ProbNet probNet) {
10        return !probNet.hasConstraintOfClass(OnlyOneAgent.class);
11    }
12
13    public static boolean onlyTemporal(ProbNet probNet) {
14        return probNet.hasConstraintOfClass(OnlyTemporalVariables.class);
15    }
16
17    public static boolean onlyChanceNodes(ProbNet probNet) {
18        return probNet.hasConstraintOfClass(OnlyChanceNodes.class);
19    }
20
21    public static boolean thereAreTemporalNodes(ProbNet probNet) {
22        return probNet.getNodes().stream()
23            .anyMatch(n -> n.getVariable().isTemporal());
24    }
25
26    public static int getNumCriteria(ProbNet probNet) {
27        // Usa TreeSet para deduplicar en O(n log n), no O(n*m)
```

```

28         Set<String> names = new TreeSet<>();
29         probNet.getPotentialsByRole(PotentialRole.UTILITY)
30             .forEach(p -> names.add(p.getCriterion().toString()));
31         return names.size();
32     }
33 }

```

Listado 7: Implementación en ProbNetClassifier.

3.5. Fase 4: extracción de ProbNetAgentManager

Motivación (SRP, §1.1; Extract Class, §1.3): la gestión de agentes en ProbNet se realizaba mediante un método `modifyAgent()` que implementaba cuatro operaciones (ADD, REMOVE, RENAME, UP/DOWN) usando un `switch` largo, con la lógica de limpieza de referencias en nodos cuando se eliminaba un agente. Esta responsabilidad es completamente ajena a la lógica de inferencia y solo la usa la GUI. Además, el bug B3 (asignación de `agents = null`) vivía exactamente en ese `switch`.

Decisión: se extrajo `modifyAgent()` a `ProbNetAgentManager`, resolviendo a la vez el bug B3. El listado 8 muestra el fragmento relevante de la corrección del bug dentro de la clase extraída.

```

1 public static void modifyAgent(ProbNet probNet,
2                               StateAction stateAction,
3                               String agentName,
4                               Object[][] dataTable) {
5     List<StringWithProperties> agents = probNet.getAgents();
6     switch (stateAction) {
7         case ADD -> {
8             if (agents == null) agents = new ArrayList<>();
9             agents.add(new StringWithProperties(agentName));
10            probNet.setAgents(agents);
11        }
12        case REMOVE -> {
13            // ANTES: agents = null <-- causaba NPE posterior
14            // AHORA: eliminar el agente y limpiar los nodos
15            agents.removeIf(a -> a.getString().equals(agentName));
16            for (Node node : probNet.getNodes(NodeType.DECISION)) {
17                if (agentName.equals(node.getAgent())) {
18                    node.setAgent(null);
19                }
20            }
21            probNet.setAgents(agents); // lista vacia, nunca null
22        }
23        // ... casos RENAME, UP, DOWN ...
24    }
25 }

```

Listado 8: Bug B3 corregido en `ProbNetAgentManager.modifyAgent()`.

3.6. Fase 5: extracción de ProbNetPotentialQueries

Motivación (SRP, §1.1; DRY, §1.1; Extract Class, §1.3): ProbNet contenía cuatro métodos que realizaban consultas complejas sobre los potenciales de la red: `getProbPotentials()`, `getUtilityPotentials()`, `tableProjectPotentials()` y `buildMarkovDecisionNetwork()`. Los dos primeros seguían el mismo patrón: obtener el nodo de la variable, recopilar vecinos, iterar potenciales y filtrar. Solo difería el predicado de filtro, violando el principio DRY.

Además, estos métodos realizan trabajo que va más allá del simple acceso a datos: construyen objetos, filtran y transforman potenciales. Ese nivel de lógica no pertenece al contenedor de la

red sino a una capa de consulta sobre él.

Decisión: se extrajeron los cuatro métodos a ProbNetPotentialQueries, unificando el patrón de los dos métodos de consulta por tipo. ProbNet conserva las delegaciones.

```
1 public final class ProbNetPotentialQueries {
2
3     private ProbNetPotentialQueries() {}
4
5     // Un unico metodo privado con el patron comun:
6     private static List<Potential> getPotentialsByRole(
7         ProbNet probNet, Variable variable, PotentialRole role) {
8         List<Potential> result = new ArrayList<>();
9         Node variableNode = probNet.getNode(variable);
10        if (variableNode == null) return result;
11        Set<Node> neighbors = new HashSet<>(probNet.getNeighbors(variableNode));
12        neighbors.add(variableNode);
13        for (Node neighbor : neighbors) {
14            for (Potential potential : neighbor.getPotentials()) {
15                if (potential.getPotentialRole() == role
16                    && potential.getVariables().contains(variable)) {
17                    result.add(potential);
18                }
19            }
20        }
21        return result;
22    }
23
24    // Los dos metodos publicos delegan en el privado:
25    public static List<Potential> getProbPotentials(
26        ProbNet probNet, Variable variable) {
27        return getPotentialsByRole(probNet, variable,
28            PotentialRole.CONDITIONAL_PROBABILITY);
29    }
30
31    public static List<Potential> getUtilityPotentials(
32        ProbNet probNet, Variable variable) {
33        return getPotentialsByRole(probNet, variable,
34            PotentialRole.UTILITY);
35    }
36
37    // ...tableProjectPotentials y buildMarkovDecisionNetwork...
38 }
```

Listado 9: Unificación del patrón duplicado en ProbNetPotentialQueries.

3.7. El cambio más importante: de herencia a composición

Motivación (composición frente a herencia, §1.1; ISP, §1.1): la relación de herencia ProbNet extends Graph<Node> era el problema de acoplamiento más profundo de la clase. Significaba que ProbNet exponía toda la API interna del grafo genérico (addVertex, removeVertex y otras operaciones de bajo nivel propias de la librería JGraphT) mezclada con la API de dominio de la red probabilística. No había forma de que un algoritmo de inferencia declarase que solo necesitaba “consultar el grafo” sin depender de toda la clase concreta ProbNet.

Decisión: dos interfaces y composición interna.

Se introdujeron dos interfaces:

- **GraphNetwork:** vista de solo lectura de la topología del grafo y las variables. La implementan ProbNet y cualquier otra clase que represente una red con estructura de grafo.

- **PotentialNetwork** extends **GraphNetwork**: extiende **GraphNetwork** con acceso a potenciales. Es la interfaz mínima que necesita un algoritmo de inferencia.

ProbNet pasa de **extends Graph<Node>** a **implements PotentialNetwork** y almacena el grafo como campo privado:

```
1 // ANTES:
2 public class ProbNet extends Graph<Node> implements ... {
3     // hereda addVertex, removeVertex y toda la API de Graph
4     // No hay forma de ocultar las operaciones internas del grafo
5 }
6
7 // DESPUES:
8 public class ProbNet implements PotentialNetwork, ... {
9     // el grafo es un detalle de implementacion privado
10    private final Graph<Node> graph = new Graph<>();
11
12    // ProbNet expone solo las operaciones con semantica de red:
13    @Override
14    public List<Node> getNodes() {
15        return graph.getNodes(); // delega internamente
16    }
17
18    @Override
19    public List<Node> getChildren(Node node) {
20        return graph.getSuccessors(node);
21    }
22    // ...
23 }
```

Listado 10: Cambio de herencia a composición en **ProbNet**.

La Figura 2 ilustra cómo el nuevo diseño permite a los algoritmos declarar dependencias mínimas.

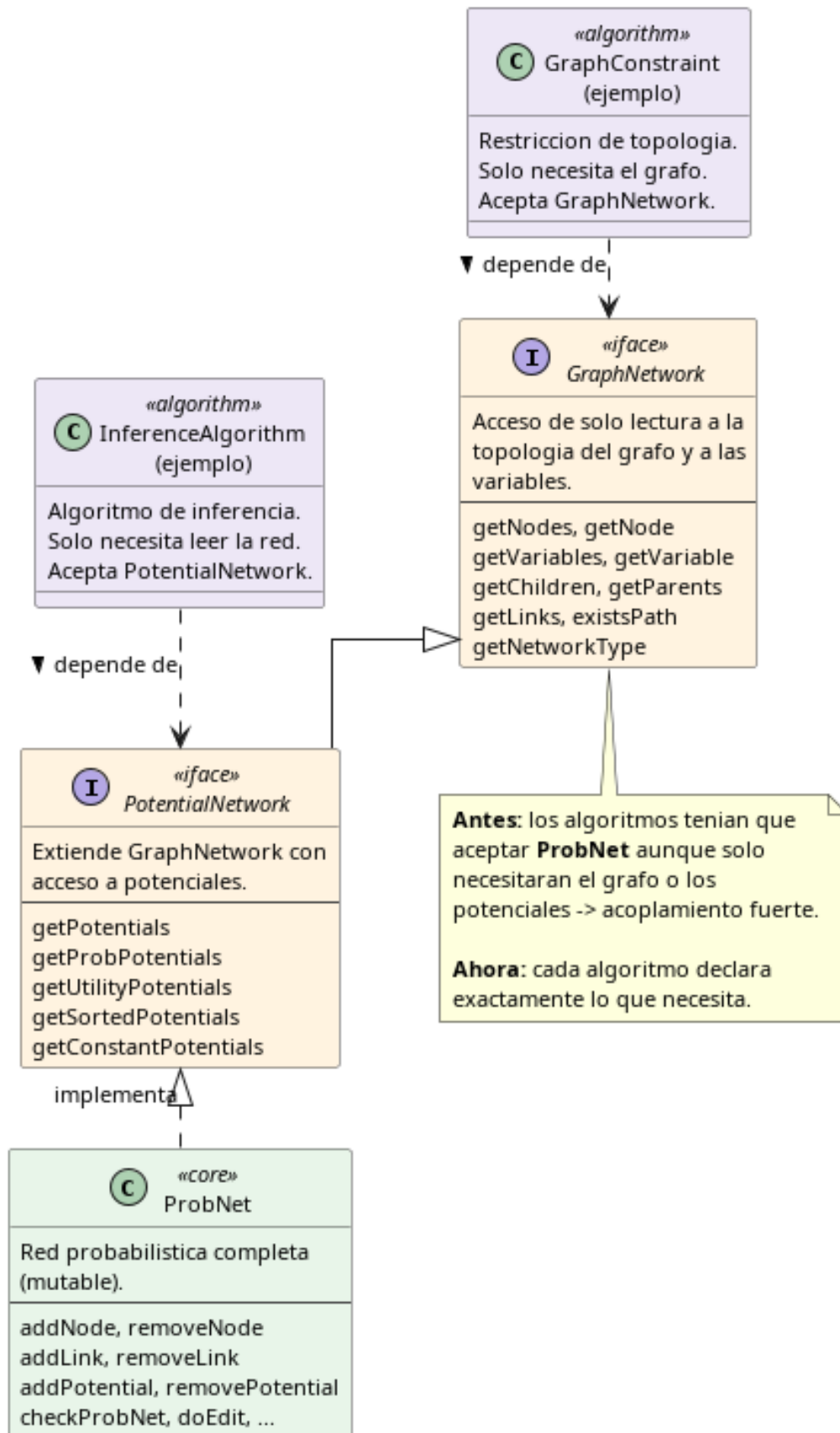


Figura 2: Jerarquía de interfaces. Los algoritmos que solo necesitan el grafo dependen de GraphNetwork; los que necesitan potenciales, de PotentialNetwork. ProbNet los implementa todos.

¿Por qué no se eliminó la API pública antigua? Algunos métodos de ProbNet que antes se heredaban de Graph se delegaban ahora explícitamente. Esto hace visible que son operaciones del grafo interno, no operaciones de dominio propias de la red. Con el tiempo, los llamadores pueden migrar a las interfaces. Se añadió la anotación @Deprecated a los métodos que ya no deberían usarse directamente.

Efecto en los algoritmos de restricción. Las restricciones de red (PNConstraint) declaran ahora checkProbNet(GraphNetwork net) en lugar de checkProbNet(ProbNet net). Esto redujo el acoplamiento de las 35 clases de restricción, que ahora solo dependen de la interfaz, no de la clase concreta.

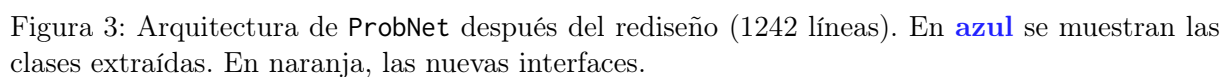
```
1 // ANTES: cada restriccion dependia de la clase concreta ProbNet
2 public class NoCycleConstraint extends PNConstraint {
3     @Override
4     public boolean checkProbNet(ProbNet net, List<Variable> variables) {
5         // ...usa net como ProbNet concreto...
6     }
7 }
8
9 // DESPUES: la restriccion solo necesita la topologia del grafo
10 public class NoCycleConstraint extends PNConstraint {
11     @Override
12     public boolean checkProbNet(GraphNetwork net, List<Variable> variables) {
13         // ...solo usa getNodes, getLinks, existsPath...
14     }
15 }
```

Listado 11: Restricciones usando GraphNetwork en lugar de ProbNet.

4. Resultado: arquitectura después del rediseño

4.1. Diagrama de la arquitectura actual

La Figura 3 muestra el diagrama UML de ProbNet y sus clases satélite después del rediseño.



4.2. Las clases extraídas

Clase extraída	Líneas	Responsabilidad
GraphNetwork	92	Interfaz: vista de solo lectura del grafo.
PotentialNetwork	47	Interfaz: extiende GraphNetwork con potenciales.
NetworkMetadata	93	Objeto valor: todos los metadatos descriptivos.
ProbNetCopier	168	Copia superficial y profunda de la red.
ProbNetClassifier	90	Consultas de clasificación (6 métodos estáticos).
ProbNetAgentManager	87	Gestión de agentes (ADD/REMOVE/RENAME/UP/DOWN).
ProbNetPotentialQueries	134	Consultas complejas sobre potenciales.

4.3. Comparación de métricas

Métrica	Antes	Después	Cambio
Líneas de ProbNet.java	1 440	1 242	−14 %
Campos en ProbNet	16	7	−56 %
Bugs conocidos	4	1*	−75 %
Code smells	7	0	−100 %
Clases en el paquete network	—	+7	(nuevas clases satélite)
Restricciones acopladas a ProbNet	35	0	todas usan GraphNetwork

* Bug B4 (addShiftedNode() sin potenciales) permanece documentado con @ToCheck hasta que se decida el comportamiento correcto.

4.4. Qué no cambió

La API pública de ProbNet no se rompió en ningún punto: todos los métodos que existían antes siguen existiendo con la misma firma. Los 490 ficheros que referencian a ProbNet no necesitaron modificación alguna para que los tests pasaran. Esto fue posible gracias a las delegaciones que ProbNet conserva hacia las clases extraídas.

La migración progresiva está disponible: los llamadores pueden ir adoptando las nuevas clases de utilidad (ProbNetClassifier.isMultiagent(net) en lugar de net.isMultiagent()) o las interfaces (GraphNetwork en lugar de ProbNet) cuando sea conveniente, sin ninguna urgencia.

5. Orden de ejecución y riesgos

Las fases se ejecutaron con el siguiente orden y nivel de riesgo:

Fase	Descripción	Riesgo	Estado
1	Corrección de bugs B1–B3 y smells S1–S7	Bajo	Completada
	Extracción de NetworkMetadata	Bajo	Completada
	Extracción de ProbNetCopier	Bajo	Completada
3	Extracción de ProbNetClassifier	Medio	Completada
4	Extracción de ProbNetAgentManager	Bajo	Completada
5	Extracción de ProbNetPotentialQueries	Medio	Completada
	Introducción de GraphNetwork/PotentialNetwork	Alto	Completada
	Adaptar restricciones a GraphNetwork	Medio	Completada
–	Resolución de bug B4 (addShiftedNode)	Bajo	Pendiente
–	Mover reader/writer a la GUI	Alto	Pendiente

Las dos tareas pendientes son independientes del resto del rediseño. La primera requiere consultar el comportamiento esperado en la GUI. La segunda requiere un análisis de impacto en los módulos gui e io.

Referencias

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2019.
- [2] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [3] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995.
- [4] A. Hunt, D. Thomas, *The Pragmatic Programmer*, Addison-Wesley, 1999.